Different market, **same job.**

Why Lifepal's insurance marketplace maps onto Tripadvisor's travel marketplace. In Lifepal, the other side was the insurers; at Tripadvisor, it's hotels, restaurants, experiences and the booking partners behind every "check prices" click.

Both businesses sit in the middle. Lifepal helped people compare insurance and sent them on to the insurers that cover them; Tripadvisor helps travellers compare places and sends them on to the partners that sell the stay. In both, events arrive late or twice, partners describe the same thing in their own shapes, records change after the fact, people can ask to be forgotten, and partner revenue depends on every click being counted once. The data problems that follow are the ones NEXUS was built to solve.

Kafka · Flink · Spark · BigQuery · Snowflake

PRESENTED TO

BUILT IN lifepal → Tripadvisor

Nexus · Lifepal best practices

ONE FLOW, TWO BUSINESSES

Tripadvisor side from public material

01 Visit	02 Compare	03 Click	04 Change	05 Earn
Lifepal · a customer comes to compare cover	**Lifepal** · products, prices, insurer terms	**Lifepal** · a quote or application to an insurer	**Lifepal** · CDC and late insurer statuses	**Lifepal** · the insurer binds; commission
Tripadvisor · a traveller searches a city	**Tripadvisor** · places, reviews, rankings	**Tripadvisor** · "check prices" to a booking partner	**Tripadvisor** · late app events, review edits, schema changes	**Tripadvisor** · the partner pays for the click

THE CLOSEST PARALLEL: PARTNER CLICKS

a marketplace sends people to partners

LIFEPAL · PERSON → INSURER

Visitor	Compares	Applies	Decision
Comes to compare insurance, on web or app.	Products and prices from many insurers.	A quote or application goes to one insurer.	Quote, bind, renew. Statuses arrive late, for days.

TRIPADVISOR · TRAVELLER → PARTNER

Traveller	Reads	Clicks	Revenue
Searches, reads, comes back on another device.	Reviews and rankings; every view is an event.	"Check prices" sends them to a partner.	The partner pays; app events can land hours late.

THE SAME PROBLEM	HOW I HANDLED IT AT LIFEPAL	WHERE IT SHOWS HERE
Events twice, or late	Kafka + Debezium CDC, merged so the newest wins and every reload is safe to rerun.	Dedupe on id, watermarks, a side output: 112 retries dropped, 192 late events explained by the batch.
Partners in their own shapes	A config-driven framework: a new source was configuration, not code.	One contract reads schema v1 and v2; an unknown currency is a dead letter, never a guess.
Pipelines people depend on	SLAs, 500+ checks, a root-cause review after every incident; escalations −60%.	Freshness from completeness; the incident found in 29 minutes, 3 with the new alert.
One number for every team	Kimball models and a dbt semantic layer; ad-hoc requests −60%.	Gold products with a stated grain and key, rendered to BigQuery and Snowflake from one file.

Described at the level I can share publicly; I'm happy to walk through the details in conversation.

WHAT TRIPADVISOR RUNS

as the Data Platform roles list it · data here is synthetic

STREAMING

Kafka · Flink · event time · SLAs

Here: three Kafka topics, six planted problems, and a Flink 1.20 job that reaches 8,707 windows identical to the Python reference.

BATCH AND WAREHOUSES

Spark · BigQuery · Snowflake · Hive

Here: a PySpark batch that checks the stream; SQL for BigQuery and Snowflake from one contract.

LANGUAGES

Java · Python · SQL

Here: the Flink job and a plain-Java twin in Java 17, the reference in Python.

SCALE

Petabytes

of data behind GenAI, analytics and ML, as the role describes the platform.

CONCEPT MAP

same grain, different names

LIFEPAL · INSURANCE MARKETPLACE	TRIPADVISOR · TRAVEL MARKETPLACE (MY READING)	GRAIN	SHARED DATA PROBLEM
Insurer partner consumer insurers · partner files	**Booking partner** check-prices clicks · CPC	one row per click	Count every click once; revenue per partner reconciles
Product page views web and app	**Location page views** hotel · restaurant · attraction	location × hour	Late and duplicated app events
Application status late insurer updates	**Review versions** submitted · moderated · edited	one row per review, latest version	Newest version wins; a tombstone can't be undone
Insurer format changes new fields mid-year	**App release changes a field** price {amount, currency}	raw record × schema version	Read both versions as one, before anyone reads it
PII masking, role-based access customer data	**Reviewer and user identifiers** policy tags · erasure	column × role	Personal data never in the clear; erasure everywhere
Insurer and finance reporting one bind rate	**Partner and internal reporting** revenue · views	one product per audience	Many audiences, one number
RAG copilot, MCP agents 1,000+ outcomes a day	**GenAI on the platform** certified products	one certified metric	Agents read the same truth as people, with permissions

Lifepal examples are from my own work. Tripadvisor examples come from the role, public information, and the synthetic data in this project.

WHAT CARRIES OVER DIRECTLY

proven at Lifepal

Exactly once, from at-least-once

Lifepal: CDC merged so the newest wins and reloads are safe to rerun.

Here: a lost TaskManager restored from a checkpoint; the sink identical.

Reliability as a feature

Lifepal: SLAs, runbooks, RCA after every incident; escalations −60%.

Here: a postmortem written by the run, and a restart alert that pages in 3 minutes.

Spark, fast and cheap

Lifepal: nightly batch 6 h → 90 min; $11k a month against $19k.

Here: a PySpark batch, identical to the reference, that checks the stream.

One definition per metric

Lifepal: a dbt semantic layer for five teams; ad-hoc requests −60%.

Here: one contract, rendered to BigQuery and Snowflake, checked in CI.

Tests before trust

Lifepal: 500+ checks in production.

Here: 50 tests, Flink and Java parity included; 11/11 warehouse checks pass.

WHAT IS DIFFERENT, AND HOW I WOULD ADAPT

DIFFERENCE	LIFEPAL	TRIPADVISOR	HOW I WOULD ADAPT
Stream engine	Kafka + Debezium with Spark	Kafka with Flink	The semantics are already written on Flink 1.20 here; learn the team's operators, state backend and deployment
Language	Python in production	Java and Python in production	Start where a Java twin tested against a reference is enough; pair early on existing services
Warehouse	Databricks and Delta with dbt; Snowflake at Deloitte	Snowflake, BigQuery and Hive	One contract for all of them; learn BigQuery's cost model (bytes scanned, slots) first
Scale	A high-traffic national marketplace	Petabyte scale, every data user in the company	Same contracts and tests; partitioning, clustering and incremental builds from day one
Who reads it	Internal teams and insurer partners	Analytics, ML, GenAI and every product team	Products with an owner, a grain, a published freshness and an SLA; breaking changes need approval

From NEXUS, built in Lifepal. The practices on this page are NEXUS best practices from my production work at Lifepal. **Stated plainly.** Lifepal outcomes come from my own record of the work. The Tripadvisor side is built from outside, from the role and public information; every page view, click, review, partner and location in the demo is synthetic. No Tripadvisor systems or data were used. My production streaming was Kafka with Spark, not Flink; BigQuery I haven't run in production.
Open questions for week one: which pipelines page most and why, how freshness SLAs are measured today, how Snowflake, BigQuery and Hive split across teams, and how streaming splits between Flink and Spark.

R. FIRMAN INSAN MUHAMMAD (VIM) · SENIOR DATA ENGINEER · LONDON · FIRMAN.INSAN.M@GMAIL.COM